

Linux Disk Space Management Guide

Interactive Disk Analysis, File Management, Cleanup, and Automation Using TUI Tools & Cron

Purpose

Disk space issues are a common cause of production failures. Regular monitoring and automated cleanup help keep servers healthy and reliable. Growing logs and unused files can be monitored, and automation helps prevent storage issues and service interruptions.

Context

Disk space issues are a common cause of production incidents. Regular monitoring and automated cleanup help maintain server performance and prevent downtime. Logs, Docker resources, and temporary files accumulate over time, consuming valuable disk space.

Real-World Example

- A production Linux server reaches 95% disk usage, causing application slowdowns and deployment failures.
- After analyzing disk usage, cleaning unnecessary files, and automating cleanup, the server runs smoothly and reliably

Solution

Use interactive terminal tools to identify and remove unnecessary files. Automate regular cleanup with Cron to keep production servers clean and reliable

Monitor → Analyze → Manage (TUI) → Clean → Automate

Business Outcome

Improves production reliability by preventing disk space issues before they impact applications and services.

Reduces operational effort through automated cleanup, allowing teams to spend less time on manual maintenance.

Minimizes downtime and improves performance, resulting in more stable servers and smoother deployments.

NOTE: Always validate and test cleanup tasks before automating them in production to prevent unexpected issues and data loss.

gdu

sudo gdu /

Features: A fast, interactive Linux disk analyzer that quickly identifies the **largest files and directories consuming disk space**.

- Written in Go.
- Often faster than **ncdu**, especially on large filesystems. **And we can delete File and DIR**
- Interactive interface.

Best for

- Very large servers.
- Fast scanning of millions of files.

Navigate directories using arrow keys. **sudo gdu /**

```
gdu - Use arrow keys to navigate, press ? for help

Scanning...

Total items: 782,493, size: 83.7 GiB, elapsed time: 23s
Current item: /var/lib/docker/rootfs/overlays/.../integrate-hubot

--- / ---
77.2 GiB | /var
6.2 GiB | /home
5.9 GiB | /usr
4.0 GiB | swapfile
1.2 GiB | /snap
522.4 MiB | /opt
188.2 MiB | /boot
67.3 MiB | /root
9.3 MiB | /etc
104.0 KiB | /tmp
4.0 KiB | /srv
4.0 KiB | /sbin.usr-is-merged
4.0 KiB | /mnt
4.0 KiB | /media
4.0 KiB | /lost+found
4.0 KiB | /lib.usr-is-merged
4.0 KiB | /bin.usr-is-merged
0 B | sbin
0 B | lib64
0 B | lib
0 B | bin

Total disk usage: 95.2 GiB Apparent size: 92.5 GiB Items: 1260009 Sorting by: size desc
```

Now we can navigate, now we delete files and folders, and

Now I can navigate and delete the unused files (OR) folders shown in the image below

```
ubuntu@ip-172-31-36-199: ~
gdu - Use arrow keys to navigate, press ? for help
--- /var/lib/docker/volumes/single-node_wazuh_queue/_data/vd/feed ---
128.2 MiB 007776.sst
93.8 MiB 009843.sst
90.0 MiB 009871.sst
87.3 MiB 009846.sst
87.3 MiB 010061.sst
83.5 MiB 009826.sst
83.3 MiB 010445.sst
79.1 MiB 010261.sst
72.2 MiB 010266.sst
68.3 MiB 009825.sst
67.6 MiB 009978.sst
66.8 MiB 010265.sst
65.4 MiB 007863.sst
64.9 MiB 007804.sst
64.9 MiB 007805.sst
64.8 MiB 010006.sst
64.8 MiB 007815.sst
64.8 MiB 010227.sst
64.8 MiB 010204.sst
64.8 MiB 010002.sst
64.7 MiB 010452.sst
64.7 MiB 010345.sst
64.7 MiB 009726.sst
64.7 MiB 007785.sst
64.7 MiB 007786.sst

Are you sure you want to delete "009843.sst"?
yes no don't ask me again
```

An interactive Linux disk analyzer that helps you quickly find the largest files and directories consuming disk space. This also clearly distinguishes it from

- **sudo gdu /:** A fast, interactive Linux disk analyzer for large files, prod best
- **sudo ncd u /:** An interactive Linux disk analyzer

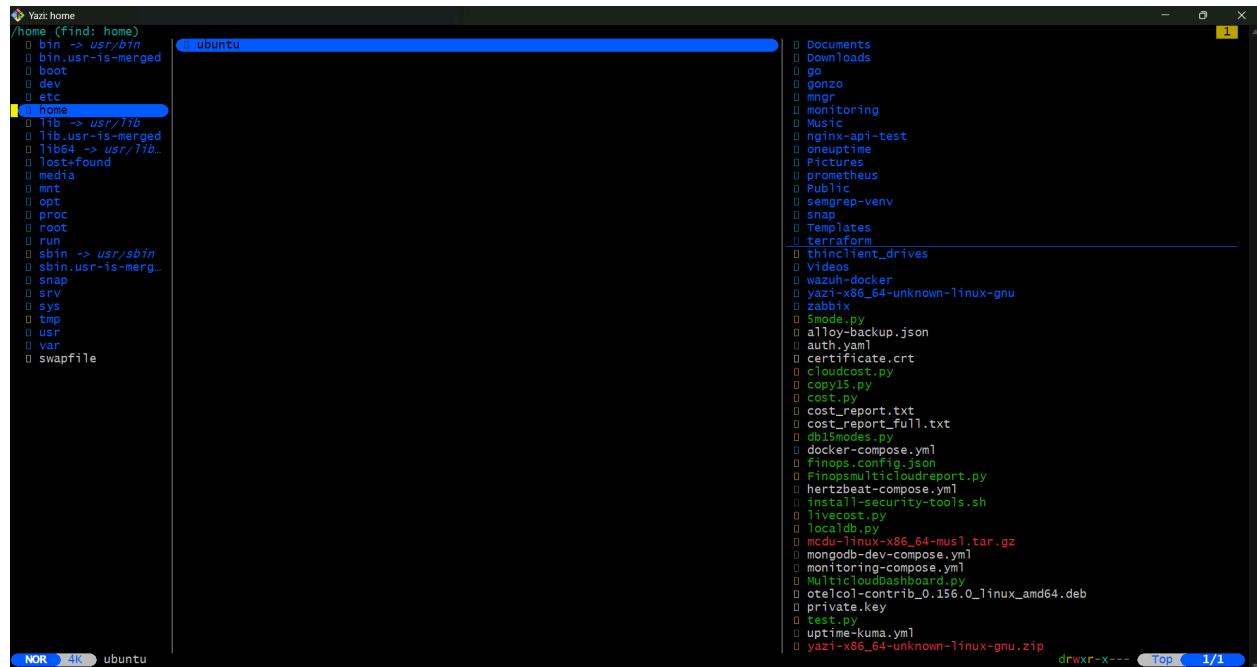
```
ncdu 1.19 - Use the arrow keys to navigate, press ? for help
/
66.4 GiB [#####] /var
6.3 GiB [##] /home
5.9 GiB [##] /usr
4.0 GiB [#] swapfile
1.2 GiB [#####] /snap
522.4 MiB [#####] /opt
188.2 MiB [#####] /boot
67.3 MiB [#####] /root
9.3 MiB [#####] /etc
3.8 MiB [#####] /run
104.0 KiB [#####] /tmp
16.0 KiB [#####] /lost+found
4.0 KiB [#####] /srv
4.0 KiB [#####] /sbin.usr-is-merged
4.0 KiB [#####] /mnt
4.0 KiB [#####] /media
4.0 KiB [#####] /lib.usr-is-merged
4.0 KiB [#####] /bin.usr-is-merged
0.0 B [#####] /proc
0.0 B [#####] /sys
0.0 B [#####] /dev
0.0 B [#####] lib64
0.0 B [#####] sbin
0.0 B [#####] lib
0.0 B [#####] bin

total disk usage: 84.5 GiB Apparent size: 128.1 TiB Items: 2112039
```

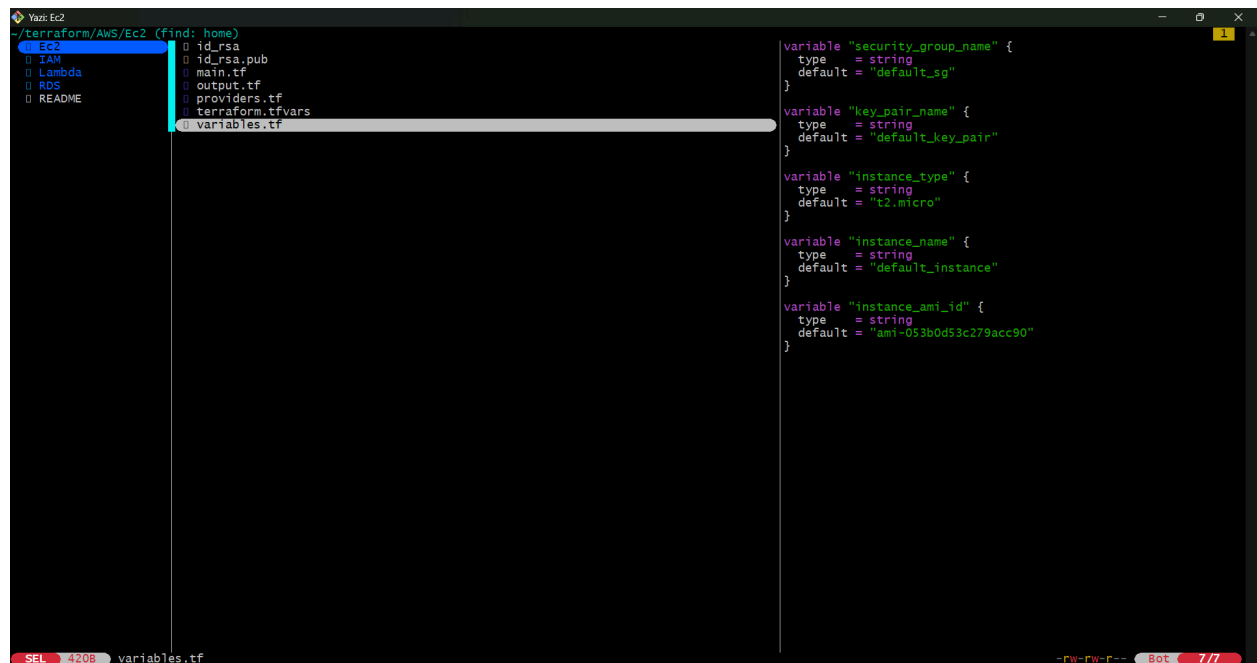
Yazi yazi

It is a terminal-based File Explorer that lets you manage files and folders quickly without leaving the command line.

yazi



Space → Select / Unselect file v → Select multiple files (Visual Mode) → aa Select All → d delete



AUTOMATION PROCESS FLOW

sudo crontab -e open cron tab

```
GNU nano 7.2 /tmp/crontab.5k3jvG/crontab *
# m h dom mon dow   command
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

#####
# Production Cleanup (Every 10 Days)
#####

#####
# Docker Cleanup
#####

# 02:00 AM - Remove stopped containers older than 10 days.
0 2 */10 * * docker container prune -f --filter "until=240h"

# 02:05 AM - Remove unused images older than 10 days.
5 2 */10 * * docker image prune -a -f --filter "until=240h"

# 02:10 AM - Remove unused Docker networks older than 10 days.
10 2 */10 * * docker network prune -f --filter "until=240h"

# 02:15 AM - Remove Docker build cache older than 10 days.
15 2 */10 * * docker builder prune -f --filter "until=240h"

# 02:20 AM - Remove all unused Docker volumes.
# (Docker does not support time filtering for volumes.)
20 2 */10 * * docker volume prune -f

#####
# Linux Cleanup
#####

# 03:00 AM - Remove unused Linux packages.
0 3 */10 * * apt autoremove -y

# 03:05 AM - Remove old APT package cache.
5 3 */10 * * apt autoclean -y

# 03:10 AM - Keep only the last 30 days of system logs.
10 3 */10 * * journalctl --vacuum-time=30d

# 03:15 AM - Delete temporary files older than 10 days.
15 3 */10 * * find /tmp -type f -mtime +10 -delete

AG Help      AQ Write Out  AW Where Is  AR Cut       AT Execute   AC Location  MU Undo      MA Set Mark  MJ To Bracket M-Q Previous
AX Exit      AR Read File  AX Replace   AU Paste     AJ Justify   AL Go To Line ME Redo      MG Copy      MQ Where was MW Next
```

SHELL=/bin/bash

PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

```
#####
# Production Cleanup (Every 10 Days)
#####
```

```
#####
# Docker Cleanup
#####
```

02:00 AM - Remove stopped containers older than 10 days.
0 2 */10 * * docker container prune -f --filter "until=240h"

02:05 AM - Remove unused images older than 10 days.
5 2 */10 * * docker image prune -a -f --filter "until=240h"

02:10 AM - Remove unused Docker networks older than 10 days.
10 2 */10 * * docker network prune -f --filter "until=240h"

02:15 AM - Remove Docker build cache older than 10 days.
15 2 */10 * * docker builder prune -f --filter "until=240h"

02:20 AM - Remove all unused Docker volumes.
(Docker does not support time filtering for volumes.) Every 10 days Removes all unused Docker volumes
20 2 */10 * * docker volume prune -f

```
#####
# Linux Cleanup
#####
```

03:00 AM - Remove unused Linux packages. Every 10 days
0 3 */10 * * apt autoremove -y

03:05 AM - Remove old APT package cache. Every 10 days
5 3 */10 * * apt autoclean -y

03:10 AM - Keep only the last 30 days of system logs.
10 3 */10 * * journalctl --vacuum-time=30d

03:15 AM - Delete temporary files older than 10 days.
15 3 */10 * * find /tmp -type f -mtime +10 -delete

sudo crontab -l → sudo systemctl status cron → journalctl -u cron -f

```
ubuntu@ip-172-31-36-199: ~
# Linux Cleanup
#####

# 03:00 AM - Remove unused Linux packages.
0 3 */10 * * apt autoremove -y

# 03:05 AM - Remove old APT package cache.
5 3 */10 * * apt autoclean -y

# 03:10 AM - Keep only the last 30 days of system logs.
10 3 */10 * * journalctl --vacuum-time=30d

# 03:15 AM - Delete temporary files older than 10 days.
15 3 */10 * * find /tmp -type f -mtime +10 -delete
ubuntu@ip-172-31-36-199:~$ sudo systemctl status cron
● cron.service - Regular background program processing daemon
   Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-08-02 15:07:07 UTC; 1h 15min ago
     Docs: man:cron(8)
  Main PID: 854 (cron)
    Tasks: 1 (limit: 8157)
  Memory: 948.0K (peak: 2.5M swap: 144.0K swap peak: 168.0K)
      CPU: 56ms
   CGroup: /system.slice/cron.service
           └─854 /usr/sbin/cron -f -P

Aug 02 15:55:01 ip-172-31-36-199 CRON[27204]: pam_unix(cron:session): session closed for user root
Aug 02 16:05:01 ip-172-31-36-199 CRON[31069]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Aug 02 16:05:01 ip-172-31-36-199 CRON[31070]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)
Aug 02 16:05:01 ip-172-31-36-199 CRON[31069]: pam_unix(cron:session): session closed for user root
Aug 02 16:15:01 ip-172-31-36-199 CRON[34858]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Aug 02 16:15:01 ip-172-31-36-199 CRON[34859]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)
Aug 02 16:15:01 ip-172-31-36-199 CRON[34858]: pam_unix(cron:session): session closed for user root
Aug 02 16:17:01 ip-172-31-36-199 CRON[35607]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Aug 02 16:17:01 ip-172-31-36-199 CRON[35608]: (root) CMD (cd / && run-parts --report /etc/cron.hourly)
Aug 02 16:17:01 ip-172-31-36-199 CRON[35607]: pam_unix(cron:session): session closed for user root
ubuntu@ip-172-31-36-199:~$ journalctl -u cron -f
Aug 02 15:55:01 ip-172-31-36-199 CRON[27204]: pam_unix(cron:session): session closed for user root
Aug 02 16:05:01 ip-172-31-36-199 CRON[31069]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Aug 02 16:05:01 ip-172-31-36-199 CRON[31070]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)
Aug 02 16:05:01 ip-172-31-36-199 CRON[31069]: pam_unix(cron:session): session closed for user root
Aug 02 16:15:01 ip-172-31-36-199 CRON[34858]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Aug 02 16:15:01 ip-172-31-36-199 CRON[34859]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)
Aug 02 16:15:01 ip-172-31-36-199 CRON[34858]: pam_unix(cron:session): session closed for user root
Aug 02 16:17:01 ip-172-31-36-199 CRON[35607]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Aug 02 16:17:01 ip-172-31-36-199 CRON[35608]: (root) CMD (cd / && run-parts --report /etc/cron.hourly)
Aug 02 16:17:01 ip-172-31-36-199 CRON[35607]: pam_unix(cron:session): session closed for user root
^C
ubuntu@ip-172-31-36-199:~$
```

Show the last page code explanation and image



PRODUCTION CRON CLEANUP

(EVERY 10 DAYS)



PURPOSE

Automatically clean Docker and Linux resources to:

- ✓ Free disk space
- ✓ Remove unused resources
- ✓ Improve system performance
- ✓ Keep the server clean and healthy
- ✓ Reduce storage usage
- ✓ Prevent disk from filling up



SHELL

`/bin/bash`

- Use the Bash shell to execute all cron jobs.
- Ensures commands run consistently.
- Supports standard Linux shell commands.



PATH

`/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin`

- Tells cron where to find Linux commands.
- Allows commands like docker, apt, journalctl and find to run without specifying full path.
- Prevents "command not found" errors.



DOCKER CLEANUP

02:00 AM – 02:20 AM

TIME	COMMAND	PURPOSE
02:00 AM	<code>docker container prune -f --filter "until=240h"</code>	• Remove stopped containers older than 10 days. • Running containers are NOT affected. • Frees Docker disk space. 
02:05 AM	<code>docker image prune -a -f --filter "until=240h"</code>	• Remove unused images older than 10 days. • Images used by containers are NOT removed. • Frees Docker storage. 
02:10 AM	<code>docker network prune -f --filter "until=240h"</code>	• Remove unused networks older than 10 days. • Clean unused networking resources. 
02:15 AM	<code>docker builder prune -f --filter "until=240h"</code>	• Remove Docker build cache older than 10 days. • Frees build cache storage. 
02:20 AM	<code>docker volume prune -f</code>	• Remove unused volumes. • Volumes attached to containers are NOT removed. • Docker does not support time filtering for volumes. 



LINUX CLEANUP

03:00 AM – 03:15 AM

TIME	COMMAND	PURPOSE
03:00 AM	<code>apt autoremove -y</code>	• Remove unused Linux packages. • Safe for production servers. • Frees disk space. 
03:05 AM	<code>apt autoclean -y</code>	• Remove old APT package cache. • Keeps the latest package cache. • Frees disk space. 
03:10 AM	<code>journalctl --vacuum-time=30d</code>	• Keep only the last 30 days of system logs. • Delete older logs. • Prevent log files from growing too large. 
03:15 AM	<code>find /tmp -type f -mtime +10 -delete</code>	• Delete temporary files older than 10 days. • Clean the /tmp directory. • Free temporary storage. 



SUMMARY

DOCKER

- ✓ Remove stopped containers
- ✓ Remove unused images
- ✓ Remove unused networks
- ✓ Remove Docker build cache
- ✓ Remove unused volumes

LINUX

- ✓ Remove unused packages
- ✓ Remove old package cache
- ✓ Keep only 30 days of logs
- ✓ Delete old temporary files



SCHEDULE



Runs every 10 days



Docker Cleanup
02:00 AM – 02:20 AM



Linux Cleanup
03:00 AM – 03:15 AM



SUITABLE FOR



Ubuntu



Docker



EC2



Linux Servers



Production
Environments



DevOps / SRE

♥ Keep your server clean, fast and reliable!

